

Day 4

Core Lessons



1. usage

`response.usage` is an object that holds the **token accounting for the entire interaction**. It contains three things: `prompt_tokens`, `completion_tokens`, `total_tokens`.

`usage` : The field on the response holding all token accounting for that request (prompt + completion + total).

2. `prompt_tokens` (Input Tokens)

Tells you **how many tokens your input prompt broke into**. The bigger the prompt or the more rare-word-heavy it is, the more prompt tokens.

`prompt_tokens` : Number of tokens your input prompt breaks into.

3. `completion_tokens` (Output Tokens)

Tells you **how many tokens the model's response was**. A detailed/longer answer = more completion tokens.

`completion_tokens` : Number of tokens the model's response breaks into.

`total_tokens = prompt_tokens + completion_tokens` — this is your total cost for one interaction. (No need to add manually; `usage.total_tokens` comes ready.)

4. `finish_reason`

Tells you **why the response stopped**:

- `"stop"` → finished naturally (the model gave everything it had to give; nothing was cut).
- `"length"` → got **cut off** in the middle because the `max_tokens` limit was hit.
- `finish_reason` : Why generation stopped — `"stop"` (natural) or `"length"` (cut off by the token limit).

5. `max_tokens`

A parameter that sets **the maximum number of tokens the model may generate** in a response. If the answer is longer than that, it gets truncated (cut). Used for cost control (e.g. a manager says “responses must be under 100 tokens”).

`max_tokens` : Parameter capping the maximum tokens the model may generate. Longer answers get truncated.

Tokens & Tokenization — Detailed Notes

1. The Core Problem: Computers Only Understand Numbers

An **LLM** is ultimately just a **computer program**. It does not understand English, Hindi, or any human language — it only understands **numbers** (and at the lowest level, only `0` and `1`).

So every interaction follows this pipeline:

```
Your language → convert to NUMBER → LLM processes → outputs a NUMBER →  
convert back to your language  
"Hello"       →      [number]       →      model       →      [number]       →  
"Hey there"
```

The model runs only on numbers and produces only numbers; those outputs are converted back to your language. **The central question: how do we convert language into numbers efficiently?**

LLM (Large Language Model): A program trained on massive amounts of text that lets you interact using everyday natural language.

Natural Language: Everyday human language (English, Hindi), as opposed to machine language.

2. Two Failed Approaches (Brief)

Approach #1 — Character-level (using ASCII): Convert each letter to a number.

- *Why it failed:* The input becomes enormous (a 100-word paragraph $\approx$ 1 lakh letters), boundaries between words are hard to detect, and it would force crippling limits on input size. Too much overhead.

Approach #2 — Word-level (dictionary IDs): Give each of the $\sim$ 6 lakh Oxford Dictionary words a unique number.

- *Why it failed:* People don't speak by the dictionary. Names (Pratyush), brands (Zomato), slang/made-up words, merged words (Alexfinn), and typos (aple) are all **out-of-vocabulary**. The number of possible "words" is almost infinite, so you can't pre-assign a number to each.

ASCII: A standard that maps each character to a fixed number.

Vocabulary: The fixed set of words/tokens a model knows.

Out-of-Vocabulary (OOV): Any word not in that fixed set (names, slang, typos, brands).

3. The Solution: Tokens

A **token** is a common, reusable unit of language — a whole word or a piece of a word — whose meaning the model already knows.

How tokens are built:

1. Scan huge amounts of text (the internet).
2. Find the **most common words / word-parts**.
3. Those common pieces become the **building blocks** = tokens.

Behavior:

- **Common words** $\rightarrow$ **single token:** the, in, walk, aeroplane each become 1 token.
- **Rare words** $\rightarrow$ **split into subword tokens:** Pratyush (rare, $\sim$ 500–1000 times on the whole internet) is **not** its own token. It's broken into known pieces, e.g. Pr + ty + ush (the model has seen Pr in Pranay, Pranjali, Pragya...). One rare word $\rightarrow$ 3 tokens.

(Intuition: like a child who learns the root play and the rule -ing, then understands playing without memorizing it whole — the model stores reusable roots + suffixes and assembles unknown words from them.)

Token: A common, reusable unit (word or word-part) whose meaning the model already knows.

Subword token: A *piece* of a word (e.g. `Pr`, `ty`, `ush`, `-ing`) used to represent rare/unknown words from known parts.

Tokenization: The process of breaking text into tokens, then converting those tokens into numbers.

4. Token Count ≠ Letter Count (Important)

Common misconception: “4 letters = 1 token” or “token length = word length.” **Wrong.** What matters is **commonness**, not length:

- `Bengaluru` (9 letters, common) → **1 token** → costs ~₹1.
- `Pratyush` (8 letters, rare) → **3 tokens** → costs ~₹3.

A word with **more letters can cost less** if it's more common.

5. Why Tokens Matter: Billing

- LLMs charge **per token, not per word**. This is why “token cost” is a real concern.
- Each LLM sets its own vocabulary size — e.g. one might extract 1 crore tokens from the internet, another only 1 lakh, and operate on that set.

6. Two Types of Tokens in Every Interaction

When you send a prompt and get a response, tokens are counted on both sides:

Prompt / Input tokens: Number of tokens your input prompt breaks into.

Completion / Output tokens: Number of tokens the model's response breaks into.

Total tokens: `prompt_tokens + completion_tokens` — the total cost of one interaction.

Example: Prompt = 100 tokens, Response = 200 tokens → **Total = 300 tokens.**

Practical relevance (manager scenario): In a company you may be told “keep the agent's responses under 100 tokens” to save money. You control this with a variable (see `max_tokens`), and you can cap the **input** the same way. You can only reason about this once you understand tokens.

7. Code Demo

Setup Warning — Never name your file `token.py`

- `token.py` is effectively a **reserved name** in Python; it collides with an internal module used by the `.env` loader.

- Symptoms: `.env` “not found” errors and a `circular import` error. The lecturer spent ~7–8 minutes debugging this.
- **Fix:** Rename to `tokens.py`, `tok.py`, or safest — `token_yourname.py` (your name will never collide with an internal file).

Reserved name: A filename that collides with a built-in/internal module, breaking imports.

Circular import: An error where files import each other, or your file shadows an internal one of the same name.

Step 1 — Send multiple prompts using a list + loop

```
from dotenv import load_dotenv
from groq import Groq

load_dotenv()
client = Groq()

prompt1 = "Hi"
prompt2 = "Explain time travel in detail"
prompt3 = "Write a 1000 word essay on Machine Learning"

# Put all three into a LIST, then LOOP over them
prompts = [prompt1, prompt2, prompt3]

for prompt in prompts:
    messages = [{"role": "user", "content": prompt}]

    response = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=messages,
    )
```

List / Array: An ordered collection holding multiple items (here, prompts).

Loop (for): Runs the same block of code once for each item in a list.

Step 2 — Print the token counts

The response object has a `usage` field with token accounting:

```
usage = response.usage
print("Prompt:           ", prompt)
print("Prompt tokens:     ", usage.prompt_tokens)      # input tokens
print("Completion tokens: ", usage.completion_tokens)  # output tokens
```

```
print("Total tokens:      ", usage.total_tokens)      # prompt +
completion
```

Step 3 — Results (actual numbers from the demo)

Prompt	Prompt (input) tokens	Completion (output) tokens
"Hi"	37	23 (short)
"Explain time travel in detail"	41	109 (detailed)
"1000 word essay on ML"	45	10,320 (very long)

Note on “Hi” = 37 tokens: You’d expect a common word to be 1 token, but **each provider (Groq here) has its own internal algorithm** (adds spaces/markers plus system overhead). “Common word = 1 token” is the *general concept*, not an exact rule. Longer/more detailed responses = more completion tokens (23 → 109 → 10,320).

8. Limiting Output: `max_tokens` and `finish_reason`

`max_tokens` — **cap the response length**

```
response = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=messages,
    max_tokens=50,      # response capped at 50 tokens
)
```

With `max_tokens=50`, the model starts answering and gets **cut off** once it hits 50 tokens.

`max_tokens` : A parameter setting the maximum number of tokens the model may generate. Longer answers get truncated.

`finish_reason` — **why the response stopped**

```
choice = response.choices[0]
print("Response:      ", choice.message.content)
print("Finish reason:", choice.finish_reason)
```

`finish_reason` : Tells you why generation stopped:

- "stop" → finished **naturally** (nothing was cut).

- "length" → **cut off** because it hit the `max_tokens` limit.

Observed behavior across values

`max_tokens = 50 :`

- "Hi" → **stop** (its natural answer was already under 50).
- "time travel in detail" → **length** (cut off).
- "1000 word essay" → **length** (cut off).
- `max_tokens = 500 :`
- Time-travel prompt → **now stop** (500 was enough).
- Essay prompt → **still length**.
- `max_tokens = 5000 :`
- All three → **stop** (nothing cut off; the model gave everything it wanted).

Quick Definitions Recap

Term	Definition
LLM	Program trained on massive text; interacted with in natural language.
Natural Language	Everyday human language vs. machine language.
ASCII	Standard mapping each character to a fixed number.
Vocabulary	The fixed set of words/tokens a model knows.
Out-of-Vocabulary (OOV)	A word not in that set (names, slang, typos).
Token	A common, reusable unit (word or word-part) the model understands.
Subword token	A piece of a word (e.g. <code>Pr</code> , <code>-ing</code>) used to build rare words.
Tokenization	Breaking text into tokens, then converting to numbers.
Prompt / Input tokens	Token count of your input.
Completion / Output tokens	Token count of the response.
Total tokens	Prompt + completion (total cost).
<code>max_tokens</code>	Parameter capping response token count.
<code>finish_reason</code>	Why generation stopped: stop (natural) / length (cut off).
Reserved name	Filename colliding with an internal module (e.g. <code>token.py</code>).
Circular import	Error from files importing each other / shadowing internal modules.

One-line summary: Computers only understand numbers, so language must be converted — character-level is too bulky, word-level breaks on unknown words, so LLMs use **tokens** (common reusable word-pieces); you're billed by **total tokens**, and you control response length with `max_tokens`, checking `finish_reason` to see if the answer finished naturally or was cut off.